

obsidian-mcp-router — quick reference

v0.60.0 · multi-vault Obsidian MCP server, shipped by its Claude Code plugin — one install = everything (48 slash commands · 43 MCP tools · 44 skills)

github.com/tboome33/obsidian-mcp-router

The router in 30 seconds

A single MCP process that knows **all your Obsidian vaults** (local + remote). Each tool takes a `vault` parameter (or uses the default). No more registering one MCP server per vault.

- **One plugin install = everything** (v0.56+): the Claude Code plugin ships *and launches* the server — declared in its root `.mcp.json` (key `router`, via ``${CLAUDE_PLUGIN_ROOT}``). A manual `~/.claude.json` entry is the dev alternative; keeping both runs **two** server processes with duplicated tools (`mcp__obsidian-router__*` vs `mcp__plugin_obsidian-router_router__*`)
- **Local + remote vaults** treated identically — drop URL + API key into the config, done
- **Cross-vault:** `vault: "*"` auto-fans-out across every vault
- **Lock mode:** restrict the session to a single vault (safety, focus) · **Auto-enrichment:** Claude suggests wiki saves at 3 natural moments, 4 modes (`ClaudeAsk` / `Hybrid` / `FullAuto` / `off`)
- **Conversion toolbox** (v0.11+): PDF / DOCX / XLSX / PPTX / image (OCR) / audio / YouTube / webpage / git repo → markdown; opt-in **Docling** high-fidelity PDF + `pdf_to_images` (the model sees a PDF page); **BM25 relevance filter** (v0.47) strips off-topic blocks before synthesis
- **Knowledge graph:** typed graph JSON + reading tours + neighbors + shortest link path; export as `llms.txt` or **OKF bundle** (Google's Open Knowledge Format v0.1, with one of the ecosystem's first validators)
- **Click-to-open:** clickable links in chat via the bridge's `GET /open/<path>` route; `open_in_obsidian` does the same server-side (no browser — ideal for Claude Desktop)
- **Vault-creation wizard** (v0.35+): `plan_vault` → adjust → `provision_vault`, defaults-first (happy path = 1 interaction)
- **Multi-tenant mode** (v0.9+, opt-in): scope an instance by env vars — vault whitelist, read-only, per-user write audit — for MCPHub/proxy deployments
- **10 hooks**, split since v0.56: 2 come active with the plugin for every installer (hot-cache load + decisions recall); the other 8 (session auto-journal, wiki autocommit, link linter, doc-drift checker, update check, hot-cache guard...) are opt-in via `node scripts/setup-vault.mjs --install-hooks`

How the pieces fit — the dependency chain

```
Obsidian ← Local REST API (community plugin) ← BRIDGE (mcp-router-bridge, v0.7.0)
          ↑ HTTP per vault (port + apiKey from the Local REST API plugin)
MCP SERVER (obsidian-mcp-router) — Node ≥ 20.18.1 process on the PC
          ↑ MCP over stdio, spawned by Claude Code
```

CLAUDE CODE PLUGIN (obsidian-router) – commands + skills + agents + hooks
· since v0.56.1 the plugin SHIPS and LAUNCHES the server itself

- **Bridge** — runs *inside* Obsidian; registers extra routes on Local REST API's HTTP server (`/search/smart` , `/templates/execute` , `/open/*` , `PUT /vault-cas/*` since 0.7.0 — atomic `ifMatch` writes —, presence heartbeat). **Optional per vault:** without it, core file CRUD still works (plain Local REST API routes), but smart search, Templater execution, click-to-open links and atomic conditional writes don't (`ifMatch` then degrades to a checked non-atomic fallback). Updates itself via BRAT from GitHub releases.
- **MCP server** — Node process spawned by Claude Code. Requires the **Local REST API** plugin in every vault (mandatory) and its registry at `~/.claude/obsidian-mcp-router/config.json` (maintained by `setup-vault.mjs`); the bridge is optional per vault (see above).
- **Claude Code plugin** — its commands and skills orchestrate the server's MCP tools; two hooks (hot-cache load, decisions recall) read vault files directly from disk.

Since v0.56.1 — the plugin ships the server: installing (or updating) the plugin installs (or updates) the server in the same move; fresh installs self-heal `node_modules` via a zero-dependency `bin/` bootstrapper. No `~/.claude.json` entry needed — that path remains for dev checkouts only (`claude --plugin-dir <checkout>` replaces the installed plugin for a session).

Vault setup

1. Have Node.js ≥ 20.18.1. Conversion backends are opt-in since v0.56 (npm `postinstall` was removed): `npm run install-markitdown` (needs Python 3.10+ on `PATH`), `npm run install-docling` — until installed, the conversion tools stay listed and return an actionable install hint
2. **Easiest path:** `/obsidian-router:meta-attach-vault` — interactive wizard that attaches a vault to a workspace (common case), bootstraps a standalone vault, or registers a remote one. Provisions plugins + scaffolds the wiki + binds `.env` + conventions picker.
3. CLI alternative:

```
npm run setup-vault -- "C:\VAULTS\MyVault"
```

⇒ installs/configures **Local REST API** + bridge plugins, writes `.env` , allocates a unique port, adds to the registry. The 2 base hooks come active with the plugin; the 8 extra hooks are opt-**in** via `node scripts/setup-vault.mjs --install-hooks` .

4. Optional for `search_smart` : **Smart Connections** + **obsidian-mcp-router-bridge** · for Templater: **Templater**
5. Verify: `/obsidian-router:meta-status` or just say "*diagnose the router*"

Configuration — environment variables (workspace `.env`)

VAULT_PATH

Path of the current vault. Auto-detection: if it matches a registered vault, that vault becomes the default. Written by `setup-vault.mjs` .

OBSIDIAN_ROUTER_DEFAULT_VAULT	Explicit override of the default vault for this process. Wins over <code>VAULT_PATH</code> and over <code>config.defaultVault</code> . Also the workspace↔vault binding used by the wiki-query-first hooks.
OBSIDIAN_ROUTER_LOCKED	Locks the router to one vault for the session. Any tool call to another vault throws. Set via <code>/obsidian-router:lock <vault> --persist</code> .
OBSIDIAN_ROUTER_AUTO_ENRICH	Wiki auto-enrichment mode: <code>ClaudeAsk</code> (default) <code>Hybrid</code> <code>FullAuto</code> <code>off</code> . Set via <code>/obsidian-router:auto-mode <Mode> --persist</code> .
OBSIDIAN_ROUTER_ALLOWED_VAULTS	Multi-tenant (v0.9+): comma-separated whitelist of vaults this instance sees. Others are skipped.
OBSIDIAN_ROUTER_READONLY	Multi-tenant: truthy (<code>1</code> / <code>true</code> / <code>yes</code> / <code>on</code>) hides the 11 write tools — <code>write_file</code> , <code>append_to_file</code> , <code>patch_file</code> , <code>set_frontmatter</code> , <code>merge_frontmatter</code> , <code>move_file</code> , <code>delete_file</code> , <code>execute_template</code> , <code>download_page_assets</code> , <code>build_wiki_graph</code> , <code>provision_vault</code> — filtered from <code>ListTools</code> AND refused at call time.
OBSIDIAN_ROUTER_USER_ID	Multi-tenant: audit trail — every successful write appends an attributed line to the vault's <code>wiki-meta/journal.md</code> .
VAULT_<NAME>	A whole vault defined in one env var (JSON: <code>name</code> , <code>baseUrl</code> , <code>apiKey</code> ...) — dashboard-editable on MCPHub deployments.
OBSIDIAN_ROUTER_NO_WATCH	Disables hot-reload of the config file (useful for debugging).
OBSIDIAN_ROUTER_NO_HOT_CACHE_LOAD • ...NO_DECISIONS_RECALL • ... NO_WIKI_AUTOCOMMIT • ... NO_SESSION_JOURNAL	Per-hook opt-outs (v0.56): disable the hot-cache load, decisions recall, wiki autocommit and session journal hooks respectively. All accept <code>1</code> / <code>true</code> / <code>yes</code> / <code>on</code> .

Important files — where things live

<code>~/.claude/obsidian-mcp-router/config.json</code>	Main router config (vaults, port registry, defaultVault, disabledVaults, remoteVaults). Hot-reload enabled.
<code>~/.claude.json</code>	Manual/dev setup only: a hand-registered server entry pointing to a checkout. Since v0.56 the plugin declares its server in its own root <code>.mcp.json</code> (key <code>router</code> , <code>\${CLAUDE_PLUGIN_ROOT}</code>); a hand-registered entry <i>plus</i> the plugin = two processes with duplicated tools (dev machines may keep both deliberately).
<code>~/.claude/settings.json</code>	The 8 opt-in hooks land here via <code>setup-vault.mjs --install-hooks</code> (the 2 base hooks ship active in the plugin's

`hooks/hooks.json` — 10 hooks total). The plugin itself is enabled **per-workspace** via `<workspace>/.claude/settings.json` — 48 commands + 44 skills $\approx$ 10k context tokens, only load them where you use Obsidian.

<code><workspace>/ .env</code>	Environment variables for the current workspace (above). Auto-loaded at router boot.
<code><vault>/CLAUDE.md</code>	Vault wiki consigne — loaded by Claude Code into context. Includes the auto-enrichment consigne (4 modes) + installed conventions.
<code><vault>/wiki-meta/{catalog,journal,hot,overview}.md</code>	Karpathy LLM-wiki scaffolds (catalog, append-only journal, hot cache, overview) — under <code>wiki-meta/</code> since v0.12 (user pages live under <code>wiki/</code>). Renamed <code>index→catalog</code> / <code>log→journal</code> in v0.58 because OKF reserves those two basenames; the old names are still read. Scaffolded by <code>/obsidian-router:wiki</code> .
<code><vault>/wiki-meta/Sessions/</code>	Per-session detail journals (auto-written by the <code>session-auto-journal</code> hook; <code>log.md</code> stays a thin index).
<code>obsidian-mcp-router/docs/</code>	<code>auto-enrichment.md</code> (4 modes + 4 placement channels), <code>features/</code> (13 prose feature pages), <code>architecture.md</code> , this PDF (EN + FR).

48 slash commands /obsidian-router:<name> · auto-trigger on natural language

(EN + FR)

 16 MCP wrappers — one per core vault tool

Categories: **discover** · **read** · **write** · **manage** · **template** · **convert**

COMMAND	EFFECT	TRIGGER PHRASE (EN/FR)
discover-list-vaults	List vaults (online/latency/lock state)	"list my vaults" / "liste mes vaults"
discover-list-files	List files in a vault directory	"list files in Sessions" / "liste les fichiers de X"
read-get	Read a file (markdown + frontmatter)	"show me X" / "montre-moi X"
read-search	Plain-text (substring) search	"grep for <text>" / "trouve <texte>"
read-search-smart	Semantic search (Smart Connections)	"find notes about X" / "trouve mes notes sur X"
read-frontmatter	Read frontmatter (object or one key)	"metadata of X" / "quel est le statut de X"
write-create-or-replace	PUT — create or replace a file	"save this as X.md" / "crée une note X"
write-append	POST — append to a file	"append to my journal" / "ajoute à X"
write-patch	Surgical PATCH (heading/block/frontmatter)	"edit X section in Y" / "édite la section X dans Y"
write-frontmatter-set	Set one frontmatter key	"tag this with X" / "passe le statut de X à closed"
write-frontmatter-merge	Set multiple keys in sequence	"on X set status=closed outcome=tp1"
manage-move	Move or rename a file	"move X to Y" / "renomme X en Y"
manage-delete	Delete a file (2-step confirm guard)	"delete X" then "yes confirm=true"
template-execute	Execute a Templater template	"run X template" / "rends Templates/X.md avec arg=v"

pdf-to-markdown	Local PDF → markdown (MarkItDown, fast plain-text)	"convert this PDF to markdown" / "convertis ce PDF"
pdf-to-markdown-docling	High-fidelity PDF → markdown (Docling, tables/layout, opt-in)	"convert with docling" / "conversion haute fidélité"

3 router-state commands (lock + auto-enrichment)

COMMAND	EFFECT	TRIGGER PHRASE (EN/FR)
lock	Restrict the session to one vault (volatile or <code>--persist</code>)	"I only want to work on X" / "verrouille sur X"
unlock	Lift the lock, restore multi-vault routing	"unlock vaults" / "déverrouille les vaults"
auto-mode	Set the auto-enrichment mode (ClaudeAsk / Hybrid / FullAuto / off)	"switch to Hybrid mode" / "passe en mode Hybrid"

7 conversational helpers (setup + diagnostic)

COMMAND	EFFECT	TRIGGER PHRASE (EN/FR)
meta-setup	Manual/dev install (clone + npm link + <code>~/.claude.json</code> entry) — normal installs get the server from the plugin since v0.56	"setup obsidian-mcp-router" / "installe le router"
meta-attach-vault	Wizard — attach a vault to a workspace (common), standalone, or remote	"attach a vault to this workspace" / "connecte mon vault distant"
meta-status	Health-check every vault with fix hints	"are my vaults reachable" / "diagnostique le router"
meta-sync-template	Propagate the reference vault's plugins/snippets/docs to vaults (picker)	"sync the template to all vaults" / "synchronise le template"
sync-from-github	Update one vault or the whole fleet straight from the GitHub skeleton (plugins, themes, snippets, docs) — no local dev repo needed	"sync my vaults from github" / "mets à jour depuis GitHub"
meta-audit-bridge-readiness	Audit click-to-open readiness (bridge, REST API, HTTP, live /open probe)	"is click-to-open ready" / "audite le bridge"

conventions

Install/remove/status/propagate
CLAUDE.md conventions cross-
vaults

"install source-type on X" / "installe
la convention X"



21 knowledge-management commands (Karpathy-style LLM-wiki)

COMMAND	EFFECT	TRIGGER PHRASE (EN/FR)
wiki	Scaffold the wiki in a vault (index, log, hot, overview)	"set up a wiki" / "scaffold un wiki"
wiki-ingest	Ingest a source → entity/concept pages + cross-refs	"absorb this article" / "ingère cette URL"
wiki-query	3-tier RAG over the wiki (no web)	"what does my wiki say about X" / "d'après mes notes ..."
wiki-lint	Health check (orphans, dead links, index drift)	"audit my wiki" / "lint le wiki"
wiki-fold	Idempotent log rollup under wiki/folds/	"fold the log" / "compacte le journal"
hot-compact	Compact an oversized hot.md back to its cache contract (verified backup first)	"compact the hot cache" / "compacte le hot"
save	File the conversation as a typed wiki note (session/decision/ADR/...)	"save this" / "sauvegarde ça"
decision-consolidate	Compress a settled decision page to its essentials; move the full deliberation history to a verified archive note	"consolidate this decision" / "consolide cette décision"
autoresearch	Autonomous web→synth→file loop	"research X online" / "fais une recherche web sur X"
canvas	Create/edit Obsidian .canvas files	"create a canvas for X" / "crée un canvas pour X"
defuddle	Strip noise from web pages before ingestion	"defuddle <url>" / "nettoie cette page"
obsidian-bases	Create/edit Obsidian .base files (database views)	"create a base for X" / "crée une base pour X"
wiki-graph	Build the typed knowledge-graph JSON → <code>wiki-meta/graph/</code> + an <code>.understand-anything/</code> copy for the Understand-Anything dashboard (deterministic, no LLM)	"build the wiki graph" / "construis le graphe"

wiki-tour	Ordered pedagogical reading tour from the link topology	"give me a tour of this vault" / "par où je commence"
wiki-neighbors	One page's neighbors — links out, backlinks, or both	"what links to X" / "voisins de X"
wiki-path	Shortest link chain between two pages	"how is X connected to Y" / "quel rapport entre X et Y"
wiki-export	Export the vault as llms.txt / llms-full.txt or an OKF bundle	"export the wiki as llms.txt" / "exporte le wiki"
okf-export	Export a wiki subset as a shareable OKF v0.1 bundle (conformance self-checked)	"export this folder as an OKF bundle" / "publie en bundle OKF"
okf-projections	Regenerate the generated OKF navigation in wiki/ (root + per-directory index.md, newest-first log.md); auto-refreshed after writes; --check = drift report	"refresh the OKF projections" / "régénère les index du wiki"
okf-check	Validate any OKF bundle against the v0.1 conformance rules	"validate this OKF bundle" / "ce bundle est-il conforme ?"
wiki-refresh-digests	Regenerate per-page digest sidecars (concepts/claims/keywords)	"refresh the digests" / "rafraîchis les digests"
who-is-speaking	Identify the family member in a shared vault, lock routing per member	"who is speaking" / "c'est Karine"

The 43 MCP tools — what Claude actually calls under the hood

<code>list_vaults · list_files</code>	Discovery. Vault catalog with online status + latency (call first); directory listing.
<code>get_file · search · search_smart · get_frontmatter</code>	Reads. Full file (returns <code>contentSha256</code> — the <code>ifMatch</code> token); substring search; semantic search (Smart Connections embeddings, cosine scores); typed frontmatter. <code>vault: "*" = cross-vault fan-out.</code>
<code>write_file · append_to_file · patch_file · delete_file · set_frontmatter · merge_frontmatter · move_file</code>	Writes & file management. Create/replace; append; surgical patch by heading/block/frontmatter; guarded delete (<code>confirm: true</code>); one or many frontmatter keys; move/rename. ifMatch (v0.60): replay <code>get_file</code> 's <code>contentSha256</code> and the write is refused (409) if the file changed since you read it — atomic with bridge ≥ 0.7.0, checked fallback otherwise.
<code>execute_template</code>	Templater. Render a template, optionally write the result; args via <code>tp.mcpTools.prompt("key")</code> .
<code>lock_vault · unlock_vaults · set_auto_enrich_mode</code>	Router state. Single-vault isolation; auto-enrichment mode switch.
<code>plan_vault · provision_vault</code>	Vault provisioning (v0.35+). Read-only plan (defaults + questionnaire + warnings) → one-call creation (plugins, port, API key, wiki scaffold, registry). Local-only.
<code>pdf/docx/xlsx/pptx/image/audio_to_markdown</code>	Local conversion (Markdown — opt-in via <code>npm run install-markitdown</code> since v0.56; the tools stay listed and return an install hint when the backend is missing). Office/PDF/image OCR/audio transcription → markdown text; chain with <code>write_file</code> .
<code>pdf_to_markdown_docling · pdf_to_images</code>	High-fidelity PDF (opt-in Docling): layout + table-structure recognition. <code>pdf_to_images</code> renders pages to PNG so the model can see them (capped pages/bytes).
<code>youtube/bing_search/webpage_to_markdown · git_repo_to_markdown</code>	Remote conversion. URL → markdown (SSRF-guarded); YouTube transcripts; git repo → single markdown bundle (repomix, optional Tree-sitter compression). <code>webpage_to_markdown</code> accepts <code>relevanceQuery</code> (BM25).

<code>filter_relevant_blocks</code>	Relevance filter (v0.47). Deterministic BM25 second pass over markdown you already have — drops off-topic blocks, keeps frontmatter/headings, multiple no-op safety nets. No fetch, no LLM.
<code>extract_page_metadata</code> · <code>propose_linked_sources</code> · <code>download_page_assets</code>	Web ingestion support. Deterministic page metadata (JSON-LD/OpenGraph); scored link-following candidates; image download into the vault.
<code>get_wiki_context_pack</code> · <code>build_wiki_graph</code> · <code>build_wiki_tour</code> · <code>get_page_neighbors</code> · <code>wiki_path</code>	Context & graph. Structured context envelope for external agents; typed knowledge graph; reading tours; per-page neighbors; shortest link path.
<code>build_open_link</code> · <code>open_in_obsidian</code>	Navigation. Ready-to-paste click-to-open links; server-side open (raise Obsidian window, optional heading anchor) — no browser round-trip.

Tips · Type `/obsidian-router:` in Claude Code → autocomplete · Every command accepts a `vault` argument (wrappers fall back to the default) · `vault: "*" on search / search_smart` = cross-vault fan-out · Trigger phrases are indicative — Claude recognizes variants · Write results carry a ready-made `clickToOpenUrl` — one click opens the note in Obsidian · Full docs: `README.md`, `docs/features/` (13 pages), `docs/auto-enrichment.md` in the repo.